

HomeSphere

A MERN Based Home Services Marketplace

Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of:

[Degree / Program Name]

By:

[Student Name]

[Roll Number / Enrollment Number]

Under the guidance of:

[Guide / Supervisor Name]

Department of **[Department Name]**

[Institution / University Name]

Academic Year: **2025-2026**

Project Theme

HomeSphere is a production-ready web application for discovering, booking, and managing home services such as cleaning, cooking, plumbing, electrical repair, and other household support categories. The system combines online registered providers with an admin-managed local contact directory so that users can find help even when providers in a locality are not yet onboarded to the platform.

Technology Stack

MongoDB, Express.js, React.js, Node.js, Socket.IO, JWT authentication, Docker, Docker Compose, nginx, GitHub Actions.

Bonafide Certificate

This is to certify that the project report titled "**HomeSphere: A MERN Based Home Services Marketplace**" is a bonafide record of the project work carried out by **[Student Name]**, bearing Roll Number **[Roll Number]**, in partial fulfillment of the requirements for the award of the degree of **[Degree / Program Name]** during the academic year **2025-2026**.

The project was completed under my supervision and guidance. The work presented in this report is original to the best of my knowledge and has not been submitted elsewhere for the award of any other degree or diploma.

Guide

Name: **[Guide Name]**

Designation: **[Designation]**

Department: **[Department Name]**

Signature: _____

Date: _____

Head of Department

Name: **[HOD Name]**

Department: **[Department Name]**

Signature: _____

Date: _____

Institution Seal

[Seal / Stamp]

Acknowledgement

I express my sincere gratitude to **[Institution Name]** and the Department of **[Department Name]** for giving me the opportunity to work on this project. I am deeply thankful to my project guide, **[Guide Name]**, for continuous support, valuable suggestions, technical direction, and encouragement throughout the development of HomeSphere.

I also thank the faculty members and lab staff who supported the project by providing access to development tools, testing environments, and constructive feedback during review discussions. Their inputs helped improve the project from a simple marketplace concept into a more complete home services platform with authentication, booking, geolocation, notifications, provider scheduling, local contact support, and deployment readiness.

I would also like to thank my classmates, friends, and family for their motivation and support. Their practical feedback helped shape the user experience, especially around service search, booking clarity, and local help request features.

Finally, I acknowledge the open-source ecosystem that made this project possible. Technologies such as React, Node.js, Express.js, MongoDB, Mongoose, Socket.IO, Docker, and Vite provided a reliable foundation for implementing a scalable and modern web application.

[Student Name]

Table of Contents

Section	Title	Page
Front Matter	Title Page	1
Front Matter	Bonafide Certificate	2
Front Matter	Acknowledgement	3
Front Matter	Table of Contents	4
Front Matter	List of Figures	5
Front Matter	List of Tables	6
Front Matter	Abstract	7
Front Matter	Graphical Abstract	8
Front Matter	Abbreviations	9
Front Matter	Symbols	10
Chapter 1	Introduction	11-18
Chapter 2	Literature Survey	19-30
Chapter 3	Design Flow / Process	31-49
Chapter 4	Results Analysis and Validation	50-59
Chapter 5	Conclusion and Future Work	60-61
End Matter	References	62
Appendix	User Manual and Achievements	63-65

The report follows the arrangement specified in the provided academic format. The content is organized from project motivation and problem identification to design decisions, implementation, testing, validation, deployment, and future scope.

List of Figures

Figure No.	Figure Title	Page
1.1	High-level project context	11
1.2	Stakeholder interaction view	12
2.1	Evolution of home service discovery	20
2.2	Literature theme map	21
3.1	Concept generation matrix	32
3.2	Selected MERN architecture	39
3.3	Database entity relationship view	40
3.4	Authentication and authorization flow	41
3.5	Geo-aware service discovery flow	42
3.6	Booking lifecycle flow	44
3.7	Real-time notification flow	46
3.8	Deployment architecture using Docker	48
4.1	Backend MVC folder structure	51
4.2	Frontend page and component structure	52
4.3	Test execution summary	55
A.1	Local setup terminal commands	63
A.2	Docker Desktop deployment flow	64

Screenshots can be inserted during final formatting from the running application, including the home page, booking page, provider dashboard, admin dashboard, local contacts page, and Docker Desktop containers.

List of Tables

Table No.	Table Title	Page
1.1	Client and user needs	12
1.2	Task identification and deliverables	15
1.3	Project timeline	17
2.1	Summary of literature themes	22
2.2	Problem gaps identified from existing systems	29
3.1	Alternative design comparison	36
3.2	Feature finalization	37
3.3	Design constraints	38
3.4	Database collections	40
3.5	API module overview	49
4.1	Backend test cases	55
4.2	Deployment validation checklist	56
5.1	Future work roadmap	61
A.1	Demo login accounts	63

The tables are designed to summarize technical decisions and validation results in a compact academic format.

Abstract

Home services are a daily requirement in urban and semi-urban households, but users often face difficulty in finding trusted providers who are available at the required time and location. Traditional word-of-mouth methods do not scale, while many online marketplaces focus only on registered providers and may ignore local offline service workers. HomeSphere addresses this gap by building a full-stack MERN marketplace that supports both online provider booking and an admin-curated local contact directory.

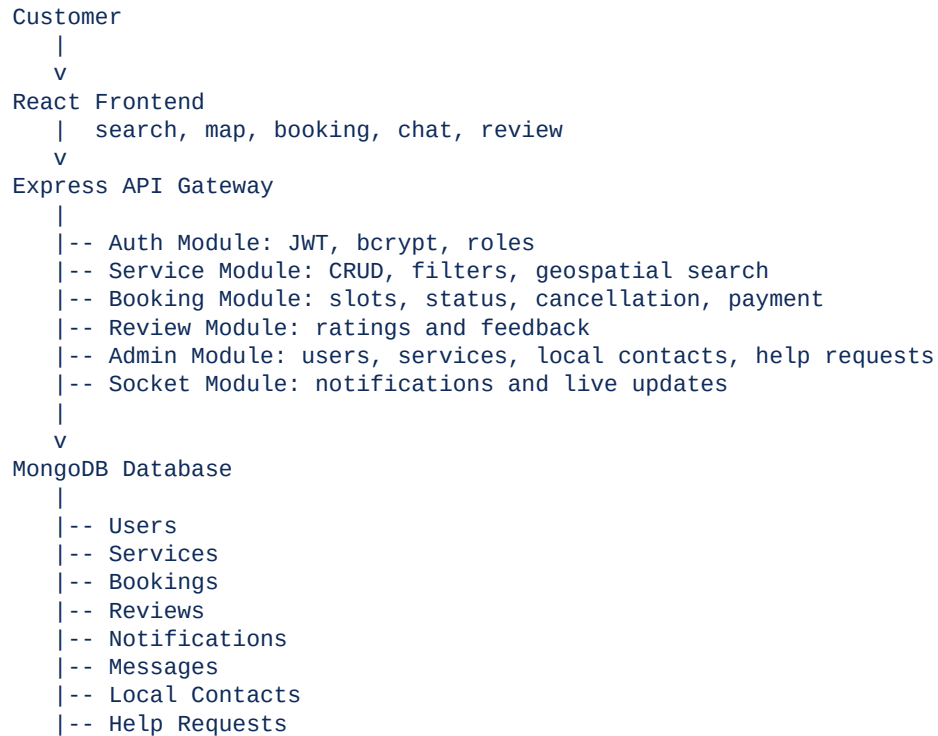
The system includes JWT based authentication, role-based access for users, providers, and administrators, service discovery by city, pincode, category, rating, price, and geolocation, provider availability scheduling, booking management, mock payment handling, reviews, chat, and real-time notifications. MongoDB geospatial queries are used to support location-aware discovery, while an AI-style ranking formula combines provider rating and distance to improve the ordering of results. Administrators can manage services, bookings, reviews, local offline contacts, and help requests raised by users.

The frontend is implemented using React with functional components and hooks. The backend follows an MVC architecture using Node.js, Express.js, and Mongoose. Socket.IO enables real-time notifications, and Docker Compose provides production-style local deployment with separate containers for MongoDB, the backend API, and the frontend nginx server. The project was validated using backend regression tests, frontend production build checks, seed data, and Docker deployment verification.

HomeSphere demonstrates how a modern web application can combine formal digital marketplace features with practical field-level local support, making the system more useful for real households and communities.

Graphical Abstract

The graphical abstract summarizes the main flow of HomeSphere from user discovery to service completion.



Project Idea in One Line

HomeSphere is a location-aware, role-based home services marketplace that supports online booking, provider scheduling, live notifications, user help requests, and offline local contact discovery.

Key Differentiator

The project does not treat the marketplace as only a list of registered providers. It also allows administrators to collect verified field data for local providers who are not yet registered, making the platform useful even in areas where the formal provider supply is still growing.

Abbreviations

Abbreviation	Full Form / Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration and Continuous Deployment
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
MVC	Model View Controller
ODM	Object Data Modeling
REST	Representational State Transfer
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
WS	WebSocket
OSM	OpenStreetMap
RBAC	Role-Based Access Control

These abbreviations are used throughout the report to describe architecture, implementation modules, and validation methods.

Symbols

Symbol	Meaning
lat	Latitude coordinate
lng	Longitude coordinate
d	Distance between user and provider
r	Provider rating
S	Final service ranking score
P	Service price
T	Selected time slot
U	User entity
A	Administrator entity
Pr	Provider entity

The ranking function used by HomeSphere can be represented conceptually as:

$$S = 0.65 * \text{normalizedRating} + 0.35 * \text{normalizedDistanceScore}$$

Here, the rating component rewards trusted providers, while the distance component improves local relevance. The final score is used for ordering search results, not for making irreversible decisions.

Chapter 1: Introduction

HomeSphere is a full-stack web application designed to solve a common household problem: finding reliable local service providers quickly and confidently. Users often need electricians, plumbers, cleaners, cooks, and other professionals at short notice. However, the existing process can be fragmented. A user may ask neighbors, call multiple contacts, search online, compare ratings, and still not know whether the provider is available at the required time.

The project introduces a digital marketplace model where users can discover services, view provider details, book available slots, track booking status, make a mock payment, chat with providers, receive notifications, and submit reviews after service completion. Providers can manage services and availability. Administrators can manage platform data, monitor bookings, and maintain a directory of local offline contacts for areas where providers may not yet be onboarded.

Figure 1.1: High-Level Project Context

```
Home need -> Search nearby providers -> Book slot -> Track work -> Review service
      |
      +--> If online provider unavailable, use admin verified local contacts
```

This dual approach makes HomeSphere different from a basic booking app. It supports both formal marketplace transactions and practical local discovery.

Identification of Client and Need

The primary client group for HomeSphere is household users who require trusted local service professionals. The secondary client group consists of service providers who need a simple way to list their skills, manage booking slots, and receive customer requests. The third stakeholder group is the administrator team, which manages quality, verifies local contacts, and handles help requests.

Table 1.1: Client and User Needs

Stakeholder	Need	HomeSphere Response
User	Find reliable service providers nearby	Location-aware service listing and ranking
User	Avoid calling many providers manually	Search, booking, status tracking, and chat
User	Get help when no online provider fits	Local contacts directory and help request form
Provider	Manage services and availability	Provider dashboard and scheduling
Provider	Receive booking updates	Real-time notifications and booking dashboard
Admin	Maintain trust and platform data	Admin dashboard for users, services, bookings, reviews
Admin	Support field-collected local providers	Local contact management module

Figure 1.2: Stakeholder Interaction View

```
User <-> Booking <-> Provider
User <-> Help Request <-> Admin
Admin <-> Local Contacts <-> User
Admin <-> Platform Records <-> Services, Reviews, Bookings
```

Relevant Contemporary Issues

The home services domain is influenced by several contemporary issues. Users expect fast digital access, transparent pricing, location relevance, provider accountability, and simple communication. At the same time, many service workers operate offline and depend on local networks rather than formal app onboarding.

HomeSphere addresses these issues through a hybrid design. Online providers can register, list services, and accept bookings. Administrators can also add field-verified local contact information for offline providers. This is important in real communities where the most useful electrician or plumber may not be registered on a marketplace yet.

The project also responds to modern software expectations. Real-time updates are implemented through Socket.IO. Location-aware discovery is implemented through geospatial data. Role-based access protects dashboards. Docker deployment supports repeatable setup. A clean React UI improves user confidence and reduces friction during booking.

Contemporary Issues Considered

- Trust gap between customers and unknown service providers.
- Time wasted in manual calling and checking availability.
- Lack of reliable local discovery in smaller neighborhoods.
- Need for transparent booking and status tracking.
- Need for administrative control over field data.
- Need for deployment-ready application structure.
- Need for responsive UI across desktop and mobile.

Problem Identification

The identified problem is that users need household services, but the process of discovering, verifying, booking, and tracking those services is often inefficient. Existing informal methods are dependent on personal contacts, while purely digital systems may fail in areas where provider onboarding is incomplete.

The problem can be divided into five smaller technical and practical problems:

- **Discovery problem:** Users need to find providers by category, city, pincode, and physical location.
- **Trust problem:** Users require ratings, reviews, and provider profiles before booking.
- **Availability problem:** Users should not book a slot that the provider cannot serve.
- **Communication problem:** Users and providers need status updates and messages.
- **Local support problem:** Users need a fallback when registered providers are not enough.

HomeSphere solves these through service listing, geolocation, AI-style ranking, availability scheduling, booking status tracking, mock payment, chat, notifications, reviews, local contacts, and help requests.

Problem Statement

To design and develop a production-ready MERN based home services marketplace that enables users to discover, book, track, and review trusted service providers, while also allowing administrators to maintain local offline contact information for unregistered area providers.

Task Identification

The project tasks were identified by converting the problem statement into functional modules. Each module contributes to a complete user journey or administrative workflow.

Table 1.2: Task Identification and Deliverables

Task	Description	Deliverable
Authentication	Secure login and registration	JWT and bcrypt based auth APIs
Roles	Separate user, provider, admin access	Role middleware and protected routes
Services	Create, list, filter, and view services	Service model, APIs, and UI cards
Geolocation	Use lat/lng for nearby discovery	MongoDB geospatial queries and map UI
Ranking	Order services by rating and distance	Ranking service module
Availability	Allow providers to define slots	Provider scheduling workflow
Booking	Reserve services and track status	Booking APIs and dashboards
Payment	Simulate payment confirmation	Mock Razorpay-style paid status
Reviews	Collect feedback after completion	Review APIs and star picker
Notifications	Inform users in real time	Socket.IO notification flow
Admin	Manage platform and field data	Admin dashboard
Deployment	Run app consistently	Docker Compose and CI workflow

Project Scope

The scope of HomeSphere includes the complete development of a MERN application with backend APIs, frontend pages, database models, authentication, role-based authorization, and deployment configuration. The application is not limited to a static prototype. It includes working data models, REST APIs, Socket.IO integration, seed data, tests, and Docker-based deployment.

In Scope

- User registration and login.
- Role-based dashboards for users, providers, and admins.
- Service listing with filters and location-aware search.
- Provider profile and rating display.
- Booking creation, cancellation, status updates, and mock payment.
- Provider availability scheduling.
- Real-time notifications and booking chat.
- Review and rating system.
- Admin-managed local contacts.
- User help request submission.
- Docker Compose deployment and CI workflow.

Out of Scope for Current Version

- Real payment gateway settlement.
- Identity verification using government documents.
- Native Android or iOS mobile applications.
- Advanced machine learning model training.
- SMS or WhatsApp notification integration.
- Production cloud infrastructure with domain and SSL.

These out-of-scope items are included in the future work roadmap.

Project Timeline

The project was developed in phases so that each major technical area could be implemented, tested, and improved before moving to the next. The workflow followed an iterative model rather than a strict waterfall model, because UI and backend features evolved together based on testing.

Table 1.3: Project Timeline

Phase	Work Completed	Outcome
Phase 1	Requirement analysis and folder structure	MERN workspace with client and server
Phase 2	Authentication and roles	JWT login, registration, protected routes
Phase 3	Service and booking modules	Service discovery and booking flow
Phase 4	Reviews, payments, dashboards	User and provider workflows
Phase 5	Geolocation and ranking	Map, saved location, geospatial search
Phase 6	Notifications and chat	Socket.IO based real-time features
Phase 7	Admin and local contacts	Admin dashboard and offline provider directory
Phase 8	UX polish	Modern blue/white interface and sharper panels
Phase 9	Testing and Docker	Regression tests, build, Docker deployment

The timeline shows that the project gradually moved from base functionality to production-oriented improvements.

Organization of the Report

This report is organized into five major chapters and supporting front matter, references, appendices, user manual, and achievements.

Chapter 1 introduces the project, identifies the client needs, explains contemporary issues, defines the problem, lists the tasks, describes the project scope, and provides the development timeline.

Chapter 2 presents the literature survey. It discusses how home service discovery has evolved, reviews important themes such as digital marketplaces, geolocation, recommendation systems, real-time updates, scheduling, trust, and local service networks, and connects these themes to the HomeSphere solution.

Chapter 3 explains the design flow and process. It includes concept generation, alternative designs, design constraints, selected architecture, database design, module design, ranking logic, booking workflow, real-time notification design, and deployment plan.

Chapter 4 describes implementation, testing, result analysis, and validation. It covers backend implementation, frontend implementation, database validation, tests, Docker deployment, UI validation, and security checks.

Chapter 5 concludes the report and presents future work. The references, appendices, user manual, and achievements provide additional academic and practical support.

Chapter 2: Literature Survey

The literature survey studies existing ideas and systems related to home service marketplaces, local discovery, digital booking, trust systems, geospatial search, ranking algorithms, scheduling, real-time communication, and web application deployment. The purpose is not only to list technologies, but to understand why the selected design is suitable for the real problem.

Home service platforms typically attempt to connect customers with nearby providers. However, the quality of such systems depends on more than a service list. A useful platform must support trust, availability, clear pricing, reliable communication, and post-service feedback. Literature and industrial practice show that service marketplaces need both technical reliability and human-centered design.

In HomeSphere, the literature review directly influenced the final feature set. Geospatial search supports local relevance. Ratings and reviews support trust. Provider availability reduces failed bookings. Real-time notifications reduce uncertainty. Admin-managed local contacts address the gap between digital marketplaces and field-level service ecosystems.

The chapter is divided into themes so that each research direction can be connected to an implementation choice in the project.

Timeline of the Reported Problem

The problem of finding local services has evolved through several stages.

Figure 2.1: Evolution of Home Service Discovery



In the earliest stage, users depended almost fully on neighbors, relatives, and local shopkeepers for service recommendations. This approach created trust but lacked scale and availability transparency. Later, telephone directories and classified advertisements helped users discover more contacts but did not provide reviews or live booking.

With the growth of web and mobile platforms, users began searching online for providers. Digital marketplaces improved convenience by centralizing service information, but many systems still struggle with provider availability, location accuracy, customer trust, and incomplete coverage in smaller localities.

HomeSphere belongs to the latest stage, where a marketplace combines digital booking with local field data. It recognizes that a practical solution must serve both registered providers and offline local providers.

Bibliometric Analysis

A bibliometric view of the topic shows that the project draws from multiple domains: electronic commerce, location-based services, recommender systems, real-time web systems, trust and reputation models, and software deployment practices. In academic and industrial research, these areas are often studied separately. HomeSphere combines them into one application.

Figure 2.2: Literature Theme Map

```
Digital Marketplaces
|-- trust and reviews
|-- service discovery
|-- booking lifecycle

Location-Based Services
|-- geocoding
|-- distance calculation
|-- geospatial indexing

Recommender Systems
|-- rating score
|-- distance score
|-- blended ranking

Real-Time Systems
|-- WebSockets
|-- notifications
|-- chat

Software Engineering
|-- MVC
|-- testing
|-- Docker
|-- CI/CD
```

The concentration of themes indicates that a home services application cannot be designed as only a CRUD project. It needs an integrated engineering approach where search, trust, communication, scheduling, and operations work together.

Literature Theme Summary

Table 2.1: Summary of Literature Themes

Theme	General Observation	Application in HomeSphere
Digital marketplace	Users prefer centralized discovery and comparison	Service listing, details, booking
Trust and reputation	Ratings reduce uncertainty	Review and star rating module
Geolocation	Nearby results improve relevance	lat/lng storage and geospatial queries
Recommendation	Ranking improves decision making	65 percent rating and 35 percent distance formula
Scheduling	Availability prevents booking conflicts	Provider slot management
Real-time updates	Live feedback improves confidence	Socket.IO notifications
Local field data	Offline providers still matter	Admin-managed local contacts
DevOps	Repeatable deployment improves reliability	Docker Compose and CI

The literature themes justify the final feature set. A successful platform must support more than visual service cards. It must support the complete service lifecycle from discovery to review.

Digital Marketplaces

Digital marketplaces reduce search cost by bringing buyers and sellers into a common platform. In the home services context, the marketplace model is useful because customers can compare providers by service type, location, price, rating, and availability. Providers benefit because they receive structured booking requests instead of scattered phone calls.

However, service marketplaces are different from product marketplaces. A product can often be shipped after purchase, but a service requires time, location, human skill, and trust. This makes provider availability and customer communication essential.

HomeSphere incorporates marketplace principles through:

- Service categories such as cleaning, electrician, plumber, cooking, and other home needs.
- Service cards that display title, category, city, price, provider, distance, and rating.
- Details pages that help users evaluate a provider before booking.
- Booking forms that collect date, time, address, city, pincode, and notes.
- Provider and user dashboards that keep both parties aware of booking status.

The implementation therefore adapts marketplace ideas to the operational realities of household service delivery.

Geolocation and Location-Based Services

Location-based services are central to HomeSphere because a provider must physically travel to the customer location. A service that is highly rated but very far away may be less useful than a nearby provider with a slightly lower rating. Therefore, search must consider distance along with traditional filters.

HomeSphere stores provider locations using latitude and longitude and uses MongoDB geospatial capabilities for nearby discovery. The frontend allows the user to share device location or select an area. The last selected location is saved so that repeat visits feel faster and more personal.

The location workflow includes:

- Browser location request.
- Reverse geocoding to estimate readable city and area information.
- Saved last-used location in the browser.
- Automatic city and pincode filling when location data is available.
- Geospatial query on the backend.
- Distance display on service cards.

The UI intentionally hides raw latitude and longitude from the main search form because coordinates are not meaningful to most users. The user experience focuses on area names rather than technical numbers.

Recommendation and AI-Style Ranking

Ranking is required when multiple providers match the same search query. If results are returned randomly, the user may not see the most suitable provider first. HomeSphere uses an AI-style scoring approach that combines rating and distance.

The current ranking formula is:

`Final Score = 65 percent rating contribution + 35 percent distance contribution`

This is not a trained machine learning model, but it follows the logic of recommendation systems by combining multiple signals into a final ordering score. Rating represents trust and service quality. Distance represents convenience and travel feasibility.

The ranking service improves usability because users do not have to manually compare every provider. The top results are more likely to be both trusted and nearby.

Future versions can expand this into a richer AI model using:

- Provider completion rate.
- Cancellation rate.
- Response time.
- Repeat booking rate.
- Price competitiveness.
- Review sentiment.
- User preference history.

For the current project, the formula is transparent, explainable, and easy to validate.

Real-Time Systems and WebSockets

Real-time communication improves user confidence in booking systems. A customer should know when a booking is created, accepted, updated, cancelled, paid, completed, or reviewed. A provider should also receive updates without refreshing the page.

Traditional HTTP requests are client initiated. This means the browser asks the server for new information. WebSockets allow a persistent connection where the server can push events to the client. HomeSphere uses Socket.IO to implement this behavior.

Real-time features in the project include:

- Notification bell in the frontend.
- Server-side notification creation.
- Socket event emission to connected users.
- Booking status notifications.
- Provider dashboard updates.
- Chat messages linked to bookings.

This approach improves the perceived reliability of the application. Users feel that the system is alive and responsive instead of static.

Scheduling and Availability

Scheduling is one of the most important differences between a simple directory and a true services marketplace. A directory only tells the user whom to contact. A marketplace should help the user reserve a real time slot.

HomeSphere includes provider availability scheduling so that providers can define available windows. During booking, the backend checks whether the requested slot fits the provider's schedule. If not, the user is shown a clear error message.

Availability management reduces:

- Double booking.
- Failed service appointments.
- Manual coordination.
- User frustration.
- Provider workload confusion.

The booking model stores date, start time, end time, status, payment status, user, provider, service, address, city, pincode, and notes. This structure makes each booking traceable from creation to completion.

Payments, Trust, and Reviews

Payment and trust are closely connected in a service marketplace. Users need confidence that the booking is recorded, payment status is clear, and feedback can be submitted after the service is finished.

HomeSphere currently uses a mock Razorpay-style payment flow. This means the application marks payment as completed after the booking confirmation step, but it does not connect to a real payment gateway. This is appropriate for an academic project because it demonstrates the payment lifecycle without handling real money or sensitive payment credentials.

The review system activates after service completion. Users can rate the provider and leave written feedback. The system prevents duplicate reviews for the same completed booking, which protects rating integrity.

Trust is supported by:

- Provider profile details.
- Ratings and review counts.
- Service descriptions.
- Booking status tracking.
- Admin oversight.
- Review restriction after completion.

Future work can integrate real payment gateway verification, refund flows, invoice generation, and fraud checks.

Local Offline Contact Support

One key idea in HomeSphere is that not every useful local provider is registered on a digital platform. In many localities, service workers may be known through field networks, local shops, community groups, or referrals. A platform that ignores these workers may fail to help users in practical situations.

HomeSphere solves this by giving administrators the ability to add local helpline numbers and local provider contact information. These contacts can be filtered by city, pincode, area, and service category. Users must be logged in to view the list, which keeps the information inside the platform.

This feature improves the project in three ways:

- It supports real field data collected by administrators.
- It helps users when registered providers are unavailable.
- It creates a bridge between offline local networks and the digital marketplace.

The local contacts feature is intentionally separate from registered provider booking. It is presented as a support directory rather than a fully automated booking flow.

Gaps in Existing Systems

Many existing service discovery approaches solve only part of the problem. A phone contact list may help users call someone, but it does not provide booking, status tracking, or reviews. A pure online marketplace may provide booking, but it may not cover every locality. A static web directory may list providers but cannot confirm availability.

Table 2.2: Problem Gaps Identified from Existing Systems

Gap	Impact	HomeSphere Solution
No live availability	Users book unsuitable slots	Provider schedule validation
Weak local relevance	Far providers appear before useful nearby ones	Geospatial search and ranking
Limited trust signals	Users hesitate to book	Ratings, reviews, provider profiles
No real-time feedback	Users must refresh or guess status	Socket.IO notifications
No offline provider support	Areas with low onboarding are underserved	Admin local contact directory
No structured help request	Users cannot ask for specific local needs	Help request module
Difficult deployment	Project runs only on developer machine	Docker Compose

Problem Definition, Goals, and Objectives

Problem Definition

The project aims to design and implement a scalable home services marketplace that allows users to find trusted providers based on service category and location, book available time slots, receive real-time updates, complete payment status, communicate with providers, submit reviews, and access admin-verified local offline contacts when required.

Goals

- Build a complete MERN application with clean client and server separation.
- Implement secure authentication and role-based authorization.
- Provide location-aware service discovery.
- Improve ranking using rating and distance.
- Support booking, payment status, cancellation, chat, and reviews.
- Provide provider availability scheduling.
- Provide admin control over platform data and local field contacts.
- Deploy using Docker for repeatable execution.

Objectives

The objectives are measurable through completed modules, API behavior, frontend workflows, test results, and deployment validation. The project is successful if a user can register, search, book, pay in mock mode, receive updates, track booking status, chat, review completed work, and use local contact support when needed.

Chapter 3: Design Flow and Process

The design process for HomeSphere followed a practical engineering approach. The goal was not only to build visible pages, but to design a system where each feature had a clear responsibility and could be tested independently. The project uses the MERN stack because it supports rapid frontend development, REST API implementation, flexible MongoDB documents, and JavaScript across the full stack.

The design flow included the following stages:

- Requirement identification.
- Concept generation.
- Alternative design analysis.
- Feature selection.
- Constraint analysis.
- Architecture design.
- Database design.
- API design.
- Frontend workflow design.
- Testing and deployment planning.

The final design uses an MVC backend, React frontend, MongoDB database, Socket.IO real-time layer, Docker deployment, and GitHub Actions CI. Each module supports one or more user journeys.

Concept Generation

Concept generation began with the question: how can a user find and book home services in a way that is simple, trustworthy, and practical for real local areas? Several concepts were considered before selecting the final design.

Figure 3.1: Concept Generation Matrix

- Concept 1: Static local directory
Low complexity, but no booking or tracking
- Concept 2: Basic online marketplace
Supports booking, but weak local fallback
- Concept 3: Full MERN marketplace with admin field directory
Supports booking, trust, ranking, live updates, and offline contacts

The selected concept is the third option. It balances digital marketplace automation with real-world local support. This is important because a home services platform must work even when local provider onboarding is still incomplete.

The concept also supports future growth. More categories, providers, payment integrations, and analytics can be added without changing the core architecture.

Alternative Design 1: Static Directory System

The first design alternative was a static or semi-static local service directory. In this model, users could search for service provider contact numbers by city, pincode, or category. Administrators could add or edit contacts, but there would be no online booking lifecycle.

Advantages

- Simple to build and deploy.
- Useful in areas where providers are mostly offline.
- Low operational complexity.
- No payment or scheduling complexity.

Disadvantages

- No booking confirmation.
- No provider availability validation.
- No real-time updates.
- No online reviews after service completion.
- No structured provider dashboard.
- Users still need to call providers manually.

This design was rejected as the primary architecture because it did not satisfy the full project objective. However, part of the idea was retained as the local contacts module because offline provider discovery is still valuable.

Alternative Design 2: MERN Marketplace

The second design alternative was a standard MERN marketplace with registered users and registered providers. Users could search services, book providers, and submit reviews. Providers could manage services and bookings. Administrators could manage platform data.

Advantages

- Supports complete online booking.
- Provides dashboards for different roles.
- Enables reviews, ratings, and structured status tracking.
- Fits well with REST APIs and React pages.
- Can be deployed using Docker.

Disadvantages

- If few providers are onboarded in a locality, users may not find help.
- It may ignore useful offline service workers.
- It requires providers to register before appearing in the system.

This design was strong, but incomplete for the local support goal. Therefore, it became the base of the final design and was extended with admin-managed local contacts and help requests.

Alternative Design 3: Mobile-First Native System

The third design alternative was a mobile-first native application supported by a backend API. It could provide a polished mobile experience, device-level location integration, push notifications, and native performance.

Advantages

- Strong mobile user experience.
- Better access to device notifications and GPS.
- Familiar app behavior for end users.

Disadvantages

- Higher development effort because separate Android and iOS applications may be needed.
- Longer testing cycle.
- App store deployment complexity.
- Not ideal for a limited academic project timeline.
- Web access from desktop would still require another interface.

This option was rejected for the current version because the MERN web application provides faster development, easier demonstration, and cross-device access through the browser. A native mobile app is recommended as future work.

Alternative Design Comparison and Selection

Table 3.1: Alternative Design Comparison

Criteria	Static Directory	MERN Marketplace	Native Mobile App	Selected Hybrid MERN
Booking support	No	Yes	Yes	Yes
Local offline contacts	Yes	No	Optional	Yes
Development speed	High	Medium	Low	Medium
Real-time updates	No	Yes	Yes	Yes
Geospatial search	Limited	Yes	Yes	Yes
Admin control	Basic	Strong	Strong	Strong
Deployment simplicity	High	High	Medium	High
Academic demonstration value	Medium	High	High	Very High

The selected design is a hybrid MERN marketplace with an admin-managed local contacts directory. It provides the strongest balance between practical usefulness, technical depth, implementation feasibility, and future expandability.

Feature Finalization

After comparing alternatives, the project features were finalized based on user value, technical feasibility, and report requirements.

Table 3.2: Feature Finalization

Feature	Reason for Inclusion
JWT authentication	Protect user data and dashboards
Role-based access	Separate user, provider, and admin responsibilities
Service categories	Support common household needs
Location search	Improve local relevance
MongoDB geospatial query	Accurate nearby provider matching
AI-style ranking	Sort results by rating and distance
Provider scheduling	Prevent invalid bookings
Booking lifecycle	Track request from creation to completion
Mock payment	Represent payment state safely
Reviews	Build trust and feedback loop
Socket.IO notifications	Send live status updates
Booking chat	Improve user-provider communication
Admin local contacts	Support unregistered local providers
Help requests	Allow users to raise specific local needs
Docker deployment	Support repeatable execution

Design Constraints

Table 3.3: Design Constraints

Constraint Type	Constraint	Design Response
Regulatory	Avoid real payment data in academic demo	Use mock payment status
Economic	Use free/open-source tools	MERN, Socket.IO, Docker, OSM
Environmental	Avoid unnecessary infrastructure	Local Docker deployment
Health and safety	Home service trust and accountability	Reviews, profiles, admin monitoring
Manufacturability	Code must be maintainable	MVC backend and component-based frontend
Professional	Protect user flows and data	Auth middleware and validation
Ethical	Do not expose local contacts publicly	Require login for local contact list
Social	Support offline workers	Admin field contact directory
Political	Avoid restricted third-party dependency	Mock payment and open mapping approach
Technical	Port conflicts and deployment variation	Docker backend mapped to host port 5001

The constraints helped shape a realistic system rather than an unrealistic feature list.

Selected System Architecture

The final system architecture uses a React frontend, Express backend, MongoDB database, and Socket.IO real-time layer. Docker Compose runs the full stack with separate services for the client, server, and database.

Figure 3.2: Selected MERN Architecture



Backend Architecture

The backend follows MVC principles:

- Models define database schemas.
- Controllers handle request logic.
- Routes expose REST endpoints.
- Middleware handles authentication, authorization, errors, and rate limiting.
- Services contain reusable logic such as ranking, availability, and notifications.

This separation improves maintainability and makes the code easier to test.

Database Design

MongoDB was selected because the application stores document-oriented data such as users, services, bookings, reviews, messages, notifications, local contacts, and help requests. Mongoose provides schema validation and relationship references.

Figure 3.3: Database Entity Relationship View

```
User 1---N Service
User 1---N Booking
Service 1---N Booking
Booking 1---1 Review
Booking 1---N Message
User 1---N Notification
Admin 1---N LocalContact
User 1---N HelpRequest
```

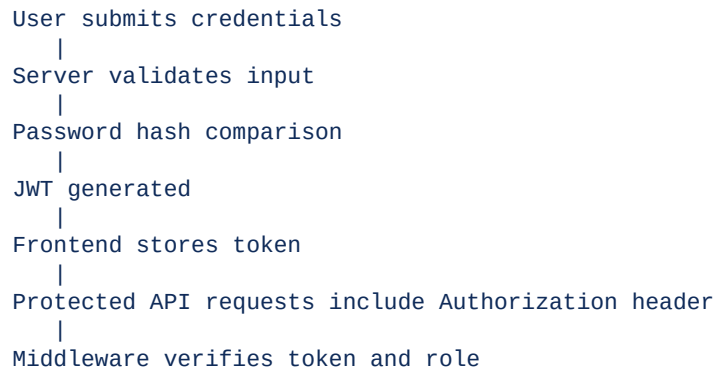
Table 3.4: Database Collections

Collection	Purpose
users	Stores customers, providers, and admins
services	Stores service listings and provider location
bookings	Stores service reservations and status
reviews	Stores ratings and feedback
notifications	Stores user notification records
messages	Stores booking chat messages
localcontacts	Stores admin-managed offline provider contacts
helprequests	Stores user raised local service needs

Authentication and Authorization Design

Authentication is implemented using JWT and bcrypt. During registration, the user's password is hashed using bcrypt before being stored. During login, the submitted password is compared with the stored hash. If the credentials are valid, the server returns a JWT.

Figure 3.4: Authentication and Authorization Flow

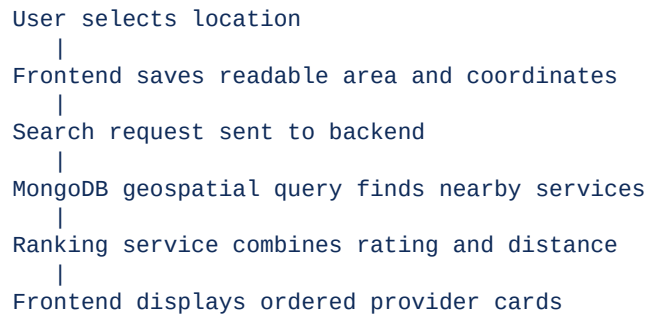


Role-based authorization separates the system into user, provider, and admin areas. Users can book and review services. Providers can manage availability and booking status. Admins can manage platform records, local contacts, and help requests.

Service Discovery and Geospatial Design

Service discovery is designed to support both traditional filters and location-aware search. Users can filter by city, pincode, category, minimum price, and minimum rating. They can also use the map/location picker to set their discovery area.

Figure 3.5: Geo-Aware Service Discovery Flow



The UI hides raw latitude and longitude from the main filter area because most users think in areas, cities, and pincodes. The map is kept smaller and cleaner so that it helps the search instead of dominating the page.

AI-Based Ranking Design

The ranking service is designed around two major signals: rating and distance. Rating reflects service quality and trust. Distance reflects convenience and local feasibility. A provider with high rating but very far distance may not be ideal, and a nearby provider with very poor rating may also not be ideal.

The selected formula gives 65 percent weight to rating and 35 percent weight to distance. This prioritizes trust while still rewarding local relevance.

$$\text{rankingScore} = \text{ratingScore} * 0.65 + \text{distanceScore} * 0.35$$

Why This Design Was Chosen

- It is explainable to users and evaluators.
- It avoids black-box decision making.
- It can be tested using known data.
- It can be extended later with real machine learning.
- It supports the project requirement for AI-based ranking.

The ranking logic is implemented on the backend so that results remain consistent regardless of frontend device or browser.

Booking, Payment, and Review Flow

The booking flow begins when a user selects a service and opens the booking page. The user enters date, start time, end time, address, city, pincode, and notes. The backend checks whether the provider is available for that slot. If the slot is valid, a booking is created.

Figure 3.6: Booking Lifecycle Flow



The payment implementation is intentionally a mock Razorpay-style flow. It demonstrates payment status changes without processing real money. The review system is restricted so that a completed booking can be reviewed only once.

Provider Availability Scheduling

Provider availability scheduling allows providers to define when they are available. This is essential because home services are time-bound. A user should not be able to book a provider at any random time.

The availability service checks whether the selected booking date and time fit the provider's configured schedule. If not, the booking API rejects the request with a clear message. This prevents invalid bookings and improves trust in the platform.

Scheduling Design Considerations

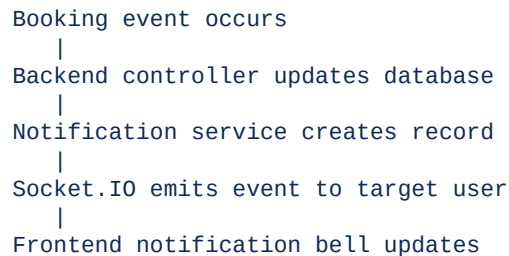
- Time slot format must be validated.
- Booking date must be stored consistently.
- Provider and service relationship must be clear.
- Availability should be checked before booking creation.
- Providers should be able to manage their schedule from the dashboard.

The current implementation supports practical slot validation. Future work can add recurring schedules, holiday blocking, buffer time between services, and calendar synchronization.

Real-Time Notification and Chat Design

Real-time communication is implemented using Socket.IO. When an important event occurs, such as booking creation, payment update, status change, cancellation, or message creation, the backend can create a notification record and emit a socket event.

Figure 3.7: Real-Time Notification Flow



Booking chat is linked to a booking so that conversation remains contextual. This is better than a general chat because messages are tied to the service request being discussed.

The design improves transparency. Users and providers do not need to repeatedly refresh dashboards to know what changed.

Admin, Local Contacts, and Help Requests

The admin module is one of the most important differentiators in HomeSphere. Administrators can manage users, services, bookings, reviews, local contacts, and user help requests.

The local contacts feature allows administrators to enter field-collected local provider information. These contacts are not full registered providers, but they can help users find offline assistance in their area. The list is visible only to logged-in users and admins.

Help requests allow users to describe a need such as "I need an electrician for wiring repair near this location." The system captures category, job title, description, city, pincode, area, and contact phone. If the user is not logged in, the platform redirects them to login before continuing.

This design supports a more community-aware marketplace. It recognizes that technology should assist real local operations instead of only serving already digitized providers.

Deployment and CI/CD Design

HomeSphere includes deployment support using Docker and Docker Compose. The full stack can be started with one command after Docker Desktop is running.

Figure 3.8: Deployment Architecture Using Docker

```
docker compose
|
|-- mongodb container: database
|-- server container: Node.js Express API
|-- client container: React build served by nginx
```

The Docker deployment maps the backend API to host port 5001 to avoid conflicts with local development port 5000. The frontend is available on host port 80. MongoDB is available on port 27017.

GitHub Actions is used for CI. The workflow installs dependencies and builds the frontend. A future improvement is to add a MongoDB service container and run backend tests automatically in CI.

Implementation Plan

The implementation plan converts design into executable modules.

Table 3.5: API Module Overview

Module	Main Responsibility
Auth	Register, login, JWT generation
Services	CRUD, filters, geospatial listing
Bookings	Create, pay, cancel, update status
Reviews	Create and list provider reviews
Notifications	Fetch and mark notifications
Messages	Booking chat
Providers	Profile and dashboard data
Admin	Manage users, services, bookings, reviews, contacts, help requests
Local Contacts	Public protected directory for offline providers
Help Requests	User local support requests

Algorithmic Flow

1. User logs in and receives JWT.
2. User selects location and search filters.
3. Backend fetches services using filters and geospatial conditions.
4. Ranking service sorts results.
5. User books provider after availability validation.
6. Provider updates status.
7. Notifications and messages keep both parties informed.
8. User reviews completed service.

Chapter 4: Results Analysis and Validation

This chapter presents the implementation results and validation of HomeSphere. The project was evaluated through functional workflows, backend regression tests, frontend production build, Docker deployment, and manual end-to-end testing.

The most important validation scenario was:

1. User creates a booking.
2. Provider sees the booking.
3. Provider updates status.
4. User receives notification.
5. User submits review after completion.

This sequence was tested successfully during development. Additional scenarios included local contact visibility, login protection for help requests, chat between booking participants, booking cancellation, and admin review of help requests.

The validation confirms that HomeSphere is not only a static UI. It is a working full-stack system with persistent database records, protected APIs, real-time behavior, and deployment support.

Modern Engineering Tools Used

The project used modern engineering tools across development, testing, deployment, and communication.

Development Tools

- Visual Studio Code for code editing.
- Node.js and npm for package management.
- Vite for fast React development.
- Express.js for backend API creation.
- MongoDB Compass for database inspection.
- Docker Desktop for container deployment.
- Git and GitHub for version control.

Backend Tools

- Mongoose for database modeling.
- bcrypt for password hashing.
- jsonwebtoken for authentication.
- Socket.IO for real-time communication.
- Express middleware for validation, authorization, rate limiting, and errors.

Frontend Tools

- React functional components.
- React hooks and context.
- Axios for API requests.
- Socket.IO client for live updates.
- CSS for responsive blue/white UI styling.

These tools support a realistic software engineering workflow.

Backend Implementation

The backend is located in the server folder and follows an MVC structure.

Figure 4.1: Backend MVC Folder Structure

```
server/src
|-- config
|-- constants
|-- controllers
|-- middlewares
|-- models
|-- routes
|-- seeds
|-- services
|-- tests
|-- utils
```

Controllers contain request handling logic. Models define MongoDB schemas. Routes map HTTP endpoints to controllers. Services contain shared business logic such as provider availability, ranking, and notification creation. Middleware protects routes, handles errors, applies role checks, and supports rate limiting.

Important backend files include:

- User, Service, Booking, Review, Notification, LocalContact, HelpRequest, and Message models.
- Auth, service, booking, review, admin, notification, provider, local contact, help request, and message controllers.
- Availability, ranking, and notification services.

This structure improves maintainability because each concern has a clear location.

Frontend Implementation

The frontend is located in the client folder and is built using React and Vite.

Figure 4.2: Frontend Page and Component Structure

```
client/src
|-- api
|-- components
|-- context
|-- pages
|-- App.jsx
|-- main.jsx
|-- styles.css
```

The application includes pages for home, service details, booking, login, register, user dashboard, provider dashboard, provider profile, user profile, local contacts, and admin dashboard. Shared components include layout, protected routes, service cards, service filters, notification bell, booking chat, and star picker.

The UI was polished to use a cleaner white and blue visual system, Open Sans font, sharper boxes, clearer partitions, and a less crowded hero area. The geolocation section was reduced in size and made easier to understand. Coordinates are hidden from the primary interface, and selected location data is remembered for repeat visits.

Database and Geospatial Validation

MongoDB validation was performed through local MongoDB and MongoDB Compass. The database stores seeded users, providers, services, bookings, contacts, and related records.

The geospatial validation focused on whether services could be searched using latitude and longitude and whether results could be ranked by distance. Seed data was used around Bengaluru so that service cards could show realistic nearby results.

Validation observations:

- Database connection succeeded using MongoDB running on localhost.
- Seed data inserted demo users and services.
- Service listing API returned results for Bengaluru.
- Geospatial search returned distance values.
- The frontend displayed service cards after backend and CORS configuration were corrected.
- The selected location could update city and pincode fields.
- Last used location was remembered by the browser.

The database design successfully supports normal marketplace records as well as local field contact records.

Testing Strategy

Testing was performed using backend regression tests and frontend build validation. The backend tests exercise important APIs and workflows using seeded test data.

The testing strategy included:

- Authentication tests.
- Booking lifecycle tests.
- Payment status update tests.
- Booking cancellation tests.
- Message creation and retrieval tests.
- Review creation and duplicate review prevention tests.
- Invalid booking slot validation.
- Admin local contact creation and public directory retrieval.
- User help request creation and admin review.

Manual testing was also performed for the browser workflows, including login, search, booking, provider dashboard, status update, notification, and review.

The combination of automated and manual testing provides confidence that the main application flows work correctly.

Backend Test Results

Figure 4.3: Test Execution Summary

```
npm test  
  
8 tests passed
```

Table 4.1: Backend Test Cases

Test Case	Result
Registers a new user	PASS
Rejects invalid email during login	PASS
Creates booking, updates payment, updates status, stores notifications	PASS
Booking participants can cancel and message each other	PASS
Completed booking can be reviewed exactly once	PASS
Rejects invalid booking slot format	PASS
Admin can add local contact and users can see directory	PASS
User can submit help request and admin can update status	PASS

The test results show that core backend behavior is functioning. These tests are especially useful because they protect the main business rules from future code changes.

Docker Deployment Validation

Docker Desktop deployment was validated using Docker Compose. The deployment starts three services: MongoDB, backend server, and frontend nginx server.

Table 4.2: Deployment Validation Checklist

Check	Expected Result	Observed Result
Docker containers start	client, server, mongodb running	Passed
Backend health endpoint	success response	Passed
Frontend loads	HTTP 200 from localhost	Passed
MongoDB reachable	Database container running	Passed
Seed data command	Demo records inserted	Passed
Port conflict handling	Backend uses host port 5001	Passed

Docker commands used:

```
docker compose up --build -d
docker compose exec server npm run seed
docker compose ps
```

The validation confirms that the project can run outside the direct local npm development workflow.

UI and UX Validation

The user interface was improved after functional testing. Early versions worked but had visual issues such as oversized sections, rounded cards that felt disconnected, visible latitude and longitude fields, and a crowded hero area. The UI was refined to feel cleaner, sharper, and easier to use.

UX improvements included:

- White background with blue accent palette.
- Open Sans font for readability.
- Sharper cards and clearer content partitions.
- Reduced geolocation section size.
- Hidden raw latitude and longitude values.
- Automatic city and pincode filling from selected location.
- Saved last-used location for repeat visits.
- Local contacts moved to a separate protected page.
- Help request form protected behind login.
- More understandable "How it works" section.

The result is a more modern interface that avoids looking like a generic marketplace clone and focuses on calm, practical booking.

Security and Validation Analysis

Security was considered at the application level. The project uses bcrypt hashing for passwords and JWT tokens for authentication. Protected routes ensure that only authenticated users can access private workflows. Role-based authorization restricts admin and provider features.

Security-related measures include:

- Password hashing before storage.
- JWT verification middleware.
- Role-based access checks.
- Protected local contacts page.
- Login requirement for help request submission.
- Backend validation for booking slot format.
- Duplicate review prevention.
- Error middleware for controlled responses.
- Rate limiting middleware for stronger backend protection.

The project still requires additional hardening before real production use. Recommended improvements include refresh tokens, secure cookie support, stronger input sanitization, audit logging, real payment verification, and deployment with HTTPS.

Data Interpretation

The test and deployment results indicate that HomeSphere meets the main functional requirements. The backend can register and authenticate users, create bookings, update booking status, manage notifications, support chat, restrict duplicate reviews, validate slots, and support admin-managed local data.

From a user perspective, the successful booking sequence proves that the application supports the full service lifecycle:

- User discovers a service.
- User books an available slot.
- Payment status is updated in mock mode.
- Provider receives and manages the booking.
- User receives status notification.
- User reviews the completed booking.

From an administrator perspective, the local contacts and help request modules prove that the system can support field operations beyond the standard provider marketplace.

The data therefore supports the conclusion that HomeSphere is a complete academic project and a strong base for production improvement.

Limitations and Deviations from Expected Results

The project achieved most planned requirements, but some limitations remain.

Current Limitations

- Payment is a mock Razorpay-style implementation, not a real payment gateway.
- CI currently focuses on frontend build and should be expanded to run backend tests with MongoDB.
- The ranking algorithm is rule-based rather than trained machine learning.
- Map behavior depends on browser location permission and geocoding availability.
- The application does not yet include SMS, WhatsApp, or email notifications.
- No native mobile application is available.
- Advanced provider verification is not implemented.
- Production-grade logging and monitoring are not yet configured.

These limitations are acceptable for the current academic scope because the core application behavior is complete. They also provide a clear path for future work.

Chapter 5: Conclusion

HomeSphere successfully demonstrates a production-oriented MERN home services marketplace. The project includes authentication, role-based access, service discovery, geolocation, AI-style ranking, provider availability, booking, mock payment, cancellation, chat, notifications, reviews, admin controls, local offline contacts, help requests, testing, and Docker deployment.

The main strength of the project is that it does not stop at a basic marketplace. It combines online registered providers with local field contacts managed by administrators. This makes the project more realistic for actual neighborhoods, where many useful providers may not yet be digitally onboarded.

The system was validated through backend tests, frontend build, Docker deployment, and manual workflow testing. The successful booking-to-review sequence shows that the application supports the complete service lifecycle. The UI improvements also make the platform easier to use and more visually consistent.

Overall, HomeSphere fulfills the project objectives and provides a strong foundation for future improvements such as real payment integration, advanced analytics, trained ranking models, mobile applications, and production cloud deployment.

Future Work

Table 5.1: Future Work Roadmap

Future Feature	Expected Benefit
Real Razorpay integration	Enable actual online payments and verification
Provider KYC verification	Improve trust and safety
SMS/WhatsApp alerts	Reach users outside the browser
Advanced AI ranking	Use booking history, response time, and review sentiment
Mobile app	Improve accessibility for daily users
Admin analytics	Track demand, category performance, and service gaps
Calendar sync	Improve provider schedule management
Refund and dispute flow	Support real payment operations
Cloud deployment with SSL	Make application publicly accessible
Automated backend CI	Run tests on GitHub Actions with MongoDB service

The most important next step is real payment integration and stronger production hardening. After that, a native mobile app and better analytics would make the platform more practical for real users and administrators.

References

1. MongoDB Documentation, Geospatial Queries and Indexes.
2. Mongoose Documentation, Schema Modeling and Query APIs.
3. Express.js Documentation, Routing and Middleware.
4. React Documentation, Components, Hooks, and State Management.
5. Socket.IO Documentation, Real-Time Bidirectional Communication.
6. JSON Web Token Introduction and Best Practices.
7. bcrypt Password Hashing References.
8. Docker Documentation, Dockerfile and Docker Compose.
9. Vite Documentation, React Build Tooling.
10. OWASP Web Security Testing and Authentication Guidance.
11. OpenStreetMap Community Documentation for Map Data Usage.
12. Razorpay Developer Documentation for Payment Gateway Concepts.

The references were used to guide the design and implementation of authentication, database modeling, APIs, frontend structure, real-time updates, deployment, and security practices.

Appendix A: User Manual - Local Setup

This section explains how to run the project locally without Docker.

Prerequisites

- Node.js installed.
- MongoDB Server running locally.
- MongoDB Compass optional for database viewing.
- Visual Studio Code recommended.

Step 1: Open Project

```
cd D:\home-services-marketplace
```

Step 2: Install Dependencies

```
npm install
npm install --workspace server
npm install --workspace client
```

Step 3: Configure Environment

Create server .env:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/home-services
JWT_SECRET=change_me
CLIENT_URL=http://localhost:5173
```

Create client .env:

```
VITE_API_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
```

Demo Accounts

Role	Email	Password
User	user@example.com	password123
Provider	provider@example.com	password123
Admin	admin@example.com	password123

Appendix B: User Manual - Running and Docker

Local Development Run

Seed the database:

```
npm run seed --workspace server
```

Run both client and server:

```
npm run dev
```

Open:

- Frontend: <http://localhost:5173>
- Backend health: <http://localhost:5000/api/health>

Docker Desktop Deployment

Start Docker Desktop, then run:

```
cd D:\home-services-marketplace  
docker compose up --build -d
```

Seed Docker database:

```
docker compose exec server npm run seed
```

Open:

- Frontend: <http://localhost>
- Backend API: <http://localhost:5001/api>
- MongoDB: <localhost:27017>

Stop containers:

```
docker compose down
```

Reset Docker data:

```
docker compose down -v
```

Achievements

The HomeSphere project achieved a broad set of technical and functional outcomes.

Functional Achievements

- Complete MERN application with client and server separation.
- JWT and bcrypt authentication.
- User, provider, and admin roles.
- Service discovery with filters.
- Map-based geolocation and saved location memory.
- MongoDB geospatial matching.
- AI-style service ranking using rating and distance.
- Provider availability scheduling.
- Booking creation, payment status, cancellation, and status tracking.
- Real-time notifications using Socket.IO.
- Booking chat between user and provider.
- Review and star rating system.
- Admin dashboard for operational control.
- Local offline contact directory for field data.
- Help request system for users needing local assistance.

Engineering Achievements

- MVC backend structure.
- React component-based frontend.
- Backend regression tests with 8 passing test cases.
- Frontend production build validation.
- Docker Compose deployment.
- README and setup documentation.
- Cleaner blue/white UI using Open Sans.

HomeSphere is therefore a complete academic software engineering project and a strong base for real-world extension.